

[1]
✓ 8s

```
!pip install -q faiss-cpu transformers torch pandas
```



23.8/23.8 MB 46.7 MB/s eta 0:00:00

[2]
✓ 2m

```
from google.colab import files

uploaded = files.upload()

... Choose files amazon_products_cleaned.csv
amazon_products_cleaned.csv(text/csv) - 37632912 bytes, last modified: 11/03/2026 - 100% done
Saving amazon_products_cleaned.csv to amazon_products_cleaned.csv
```

[3]
✓ 0s

```
import pandas as pd

file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)

print("Dataset loaded successfully")
print(df.head())
```



```
... Dataset loaded successfully
   Unnamed: 0  Product Category  Product Subcategory \
0           0      Electronics      Smartphones
1           1      Electronics      Smartphones
2           2      Electronics      Smartphones
3           3      Electronics      Smartphones
4           4      Electronics      Smartphones

   Product Name \
0  SAMSUNG Galaxy S25 Ultra Cell Phone, 512GB AI ...
1  SAMSUNG Galaxy A16 5G A Series Cell Phone, Unl...
2  Tracfone | Motorola Moto g Play 2024 | Locked ...
3  SAMSUNG Galaxy S24 FE AI Phone, 128GB Unlocked...
4  Moto G 5G | 2024 | Unlocked | Made for US 4/12...

   Product URL  Product Price \
0  https://www.amazon.com/SAMSUNG-Smartphone-Unlo...  $1,219.99
1  https://www.amazon.com/SAMSUNG-Unlocked-Smartp...  $174.99
2  https://www.amazon.com/Tracfone-Motorola-5000m...  $39.88
3  https://www.amazon.com/SAMSUNG-Unlocked-Smartp...  $498.95
4  https://www.amazon.com/Moto-Unlocked-128GB-Cam...
```

	Product	Rating	Total Ratings	\
0		4.3	247	
1		4.2	166	
2		4.3	428	
3		4.5	567	
4		4.2	1,677	

Product Image

0	https://m.media-amazon.com/images/I/61n0lmp5-...
1	https://m.media-amazon.com/images/I/71KGkQ+KOK...
2	https://m.media-amazon.com/images/I/71CxUvG46r...
3	https://m.media-amazon.com/images/I/61uakkLoHx...
4	https://m.media-amazon.com/images/I/71Sbx9zQN3...

```
[4]
TEXT_COLUMN = df.columns[0]

item_texts = df[TEXT_COLUMN].astype(str).tolist()

print("\nTotal items:", len(item_texts))
print("Sample item:", item_texts[0])
```

Total items: 42433
Sample item: 0

```
[5]
from transformers import AutoTokenizer, AutoModel
import torch
import numpy as np
import faiss

device = "cuda" if torch.cuda.is_available() else "cpu"

model_name = "sentence-transformers/all-MiniLM-L6-v2"

tokenizer = AutoTokenizer.from_pretrained(model_name)
encoder = AutoModel.from_pretrained(model_name).to(device)

encoder.eval()
```

vocab.txt: 252K? [00:00<00:00, 3.03MB/s]

tokenizer.json: 466k? [00:00<00:00, 4.36MB/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 2.84kB/s]

model.safetensors: 100% 90.9M/90.9M [00:01<00:00, 45.4MB/s]

```

encoder.eval()
vocab.txt: 232k? [00:00<00:00, 3.03MB/s]

tokenizer.json: 466k? [00:00<00:00, 4.36MB/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 2.84kB/s]

model.safetensors: 100% 90.9M/90.9M [00:01<00:00, 454MB/s]

Loading weights: 100% 103/103 [00:00<00:00, 385.67it/s, Materializing param=pooler.dense.weight]

BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status |
-----+-----+
embeddings.position_ids | UNEXPECTED |

```

Notes:

- UNEXPECTED : can be ignored when loading from different task/architecture; not ok if you expect identical arch.

```

BertModel(
  (embeddings): BertEmbeddings(
    (word_embeddings): Embedding(30522, 384, padding_idx=0)
    (position_embeddings): Embedding(512, 384)
    (token_type_embeddings): Embedding(2, 384)
    (LayerNorm): LayerNorm((384,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (encoder): BertEncoder(
    (layer): ModuleList(
      (0-5): 6 x BertLayer(
        (attention): BertAttention(
          (self): BertSelfAttention(
            (query): Linear(in_features=384, out_features=384, bias=True)
            (key): Linear(in_features=384, out_features=384, bias=True)
            (value): Linear(in_features=384, out_features=384, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (output): BertSelfOutput(
            (dense): Linear(in_features=384, out_features=384, bias=True)
            (LayerNorm): LayerNorm((384,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
    (intermediate): BertIntermediate(
      (dense): Linear(in_features=384, out_features=1536, bias=True)
      (intermediate_act_fn): GELUActivation()
    )
    (output): BertOutput(
      (dense): Linear(in_features=1536, out_features=384, bias=True)
      (LayerNorm): LayerNorm((384,), eps=1e-12, elementwise_affine=True)
    )
  )
)

```

```

        (dropout): Dropout(p=0.1, inplace=False)
    )
)
)
(pooler): BertPooler(
  (dense): Linear(in_features=384, out_features=384, bias=True)
  (activation): Tanh()
)
)

```

```

[6]
✓ 2m
def encode_texts(texts, batch_size=16):

    all_embeddings = []

    with torch.no_grad():
        for i in range(0, len(texts), batch_size):

            batch = texts[i:i+batch_size]

            tokens = tokenizer(
                batch,
                padding=True,
                truncation=True,
                return_tensors="pt"
            ).to(device)

            outputs = encoder(**tokens)

            # Mean pooling
            embeddings = outputs.last_hidden_state.mean(dim=1)

            embeddings = embeddings.cpu().numpy().astype("float32")

            all_embeddings.append(embeddings)

    return np.vstack(all_embeddings)

item_embeddings = encode_texts(item_texts)

faiss.normalize_L2(item_embeddings)

```

[6] ✓ 2m



```
    return np.vstack(all_embeddings)

item_embeddings = encode_texts(item_texts)

faiss.normalize_L2(item_embeddings)

print("\nEmbeddings shape:", item_embeddings.shape)
```

▼

...

Embeddings shape: (42433, 384)

[7] ✓ 0s



```
embedding_dim = item_embeddings.shape[1]

index = faiss.IndexFlatIP(embedding_dim)

index.add(item_embeddings)

print("FAISS index built with", index.ntotal, "items")
```

▼

...

FAISS index built with 42433 items

[8] ✓ 0s

```
def get_similar_items(item_id, k=5):

    query_vector = item_embeddings[item_id:item_id+1]

    scores, indices = index.search(query_vector, k)

    return scores[0], indices[0]
```

[9] ✓ 0s



```
query_id = 0

scores, idxs = get_similar_items(query_id, k=5)

print("\nQUERY ITEM:")
print(item_texts[query_id])

print("\nSIMILAR ITEMS:")

for score, idx in zip(scores, idxs):
    print(f"-> {item_texts[idx]} (score={score:.3f})")
```





```
query_id = 0
```

```
scores, idxs = get_similar_items(query_id, k=5)
```

```
print("\nQUERY ITEM:")
```

```
print(item_texts[query_id])
```

```
print("\nSIMILAR ITEMS:")
```

```
for score, idx in zip(scores, idxs):
```

```
    print(f"-> {item_texts[idx]} (score={score:.3f})")
```

...

```
QUERY ITEM:
```

```
0
```

```
SIMILAR ITEMS:
```

```
-> 0 (score=1.000)
```

```
-> 1 (score=0.622)
```

```
-> 100 (score=0.590)
```

```
-> 105 (score=0.569)
```

```
-> 115 (score=0.558)
```